

ModelPilot

One assistant. Continuous context. Flexible models.

Repository: github.com/LiuZongrun7/Mobile_Group_20

1 App idea and intended users

ModelPilot is an Android productivity app that helps everyday users complete tasks with different AI models in one place. It serves working professionals, creators, students and people seeking help with personal tasks. Users can write and translate text, understand documents and develop plans, with image editing as a planned extension. The app addresses three practical problems: choosing a suitable model, retaining relevant context when switching models, and understanding the cost of using AI.

Users ask directly from Chat home and organise several conversations under a project. Auto selects a suitable model for each request using task requirements, model capabilities, budget and preferences; users may also choose a model manually. ModelPilot manages conversation context independently of the provider, carrying relevant instructions, facts, files and previous results into the next request. Users can therefore change models without repeatedly explaining the same task. Insights shows the resulting usage and estimated cost.

Example: In an “Event planning” project, a user uploads a PDF brief, asks model A to summarise it, switches to model B to draft a plan, then returns to A to refine it. The event date, audience and agreed constraints remain available without re-uploading the brief. A separate poster conversation can use project instructions and files explicitly shared by the user, without mixing the full histories of both chats.

2 Related applications and the design opportunity

Application	Relevant features and source	Implication for ModelPilot
LibreChat	Self-hosted multi-provider chat with model switching, message search and agent/tool support. Source: LibreChat-AI/LibreChat; MIT.	Demonstrates a unified multi-model workspace. ModelPilot focuses on a native Android task flow linking cross-model context, per-request routing and visible cost.
Chatbox	Multi-provider client with local storage, streaming replies and rich text, including mobile versions. Community source: chatboxai/chatbox; GPL-3.0.	Informs the simple chat and model-selection experience. ModelPilot will manage context on its backend and connect each answer with its routing decision and usage record.
Cursor	AI coding product with Auto/model selection and Cloud Agents. Proprietary software: product source is not available for reuse (product terms).	Informs the direct input entry and compact Auto/manual picker. ModelPilot targets everyday non-coding tasks rather than repositories and development environments.

3 Our contribution and proposed improvements

The contribution is the integration of context continuity, task-based model selection and usage feedback in a mobile assistant. We will develop three connected capabilities:

1. Cross-model context management: preserve task instructions, important facts and file references; select and summarise relevant history for each model’s context limit. Users can inspect and correct the retained memory.
2. Explainable Auto selection: choose a model for each request, rather than permanently assigning one model to a chat. Users can see the reason for a choice or select a model themselves.
3. Task-linked usage analysis: connect answers, context summarisation and tool calls to a common usage record, so users can understand the cost of completing a task and manage their budget.

These capabilities will extend the existing Android and backend code. Our implementation work covers project/chat organisation, context assembly, provider adapters, routing rules and task-level accounting. Section 7 identifies the open-source components we will use and adapt.

4 Functional requirements and scope

Feature	Planned functionality	Development work
Chat and projects	Direct home input; create/search/move conversations; each project holds multiple chats, files and explicit instructions. Authenticated history survives restart.	Extend usage-only chat; add projects.
Auto / manual	At least two verified model APIs. Auto chooses per request; manual choice stays until changed. Reject incompatible attachments; never silently substitute a manual choice.	New router and capability registry.
Context / memory	Preserve source history; assemble instructions, versioned summaries, recent turns and file excerpts within each model's limits. Let users review/edit/reset memory; project sharing is explicit.	Add context assembly and memory controls.
Task tools	Core: text assistance and text-based PDF extraction/summarisation. Planned extension: image editing through one configured service, with preview and saved result.	New validated attachment and tool pipeline.
Insights / budget	Spend, Token and call trends; model/provider shares; App-only activity, Auto ratio and tool counts; dated price information and budget warnings/limits.	Extend existing usage/accounting modules.
Sources / controls	Label App calls and authorised imports/relay separately; deduplicate; export records; manage connections and account-scoped data. Missing values stay unknown.	Extend source contracts and privacy controls.
Explore add-ons	Source-linked news, forum posts/replies and a separate tower-defence entry. These are supplementary activities outside the assistant workflow.	Reuse existing forum and game modules.

Scope. Text/PDF tasks, cross-model context, Auto/manual selection and usage analysis form the main workflow. Image editing will be added after service integration is validated. Forum and games are supplementary features and can be developed independently; game resources remain separate from real spending and budgets. Autonomous coding, email sending and a general plugin marketplace are outside this project.

5 Proposed Android interface

The app has four top-level destinations: Chat, Insights, Explore and Me. Chat has a persistent composer and project grouping. The model picker is a bottom sheet with Auto first. Each chat's badge shows the model and brand icon used for its last completed reply. Reply details show the selection reason, model calls and tool charges. Me contains account, provider, budget and preference settings.

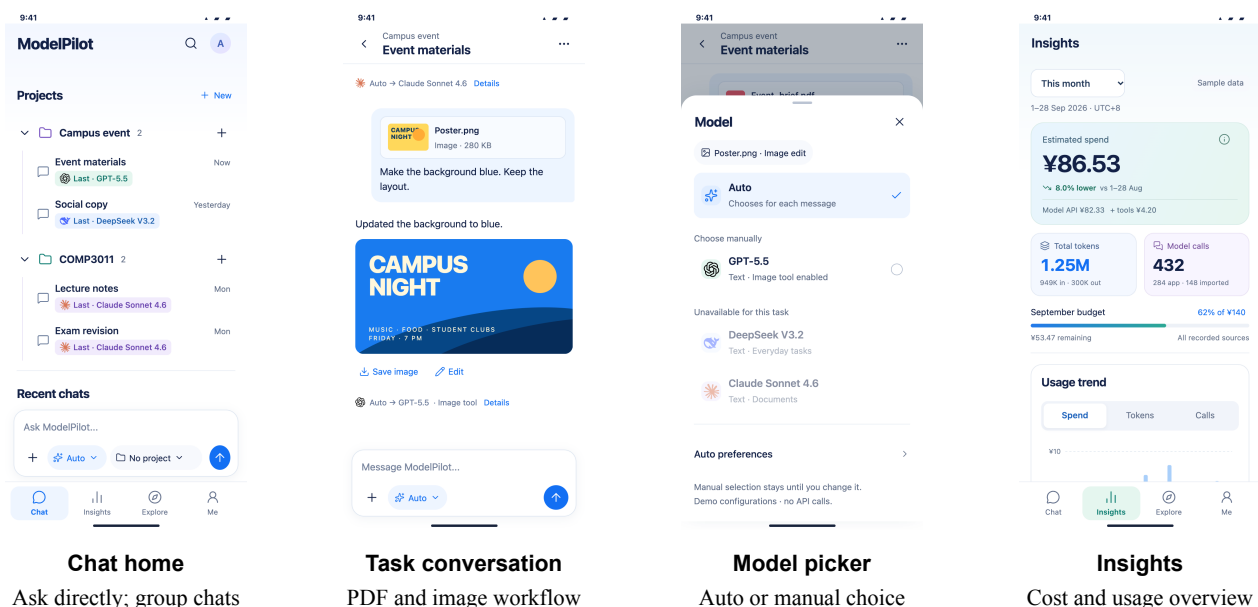


Figure 1. Proposed Chat, conversation, model-selection and Insights screens. Responses and amounts are sample data.

Mobile usability. Native file pickers, scalable text, 48dp targets and TalkBack labels. Preserve drafts and show progress/errors. Cached history remains readable offline; AI requests need connectivity.

6 Four data aspects and system responsibilities

Architecture: Android handles interaction, local caching and files. The ModelPilot backend owns authentication, history/context, routing, controlled tools and accounting. External providers perform inference and hosted image editing through APIs. We do not host model weights or require a GPU server.

Aspect	Implementation and added value
Data input	Text, project/model selection, PDF/image files, context edits, preferences, budget and authorised imports. Validate file type, size and access; clearly flag unsupported or scanned PDFs.
Processing	Match capabilities, rank feasible routes, select/compress context, run approved tools and normalise usage/cost. This produces a task result and reviewable decision, beyond API forwarding.
Storage	SQLite stores user/project/chat-scoped messages, versioned summaries, context-source IDs, routing/call/tool logs and rate versions. Protected files hold attachments/results; Room caches user-scoped history.
Output	Streamed answers, PDF notes and available image results; editable memory summaries, actual-model badges, route/usage details, charts and exports linked to source records.

Cross-model context management. The backend keeps the full conversation history and builds a smaller context for each request: project instructions, a versioned summary, recent messages and relevant file/tool excerpts. It reserves output space within the selected model's context limit. Provider adapters convert this shared record into each API's message format, allowing A–B–A switching within the same conversation. Users can inspect, correct or reset memory; editing/deleting a source invalidates affected summaries. Source links help recover details omitted by summarisation. Accounts are isolated; chats share only context explicitly selected within the same project.

Explainable Auto and controlled tools. Rules infer task needs; filter verified capabilities, access and context limits before ranking cost, measured latency and preferences. Save the selected route, policy version and reason; reserve allowance and bound steps, output, time and retries. If no route is suitable, ask the user. Tools are shared backend functions accessed through compatible adapters, not model-owned skills. PDF extraction uses pypdf; image editing requires a verified service. Uploaded content is untrusted and cannot authorise arbitrary shell/Python execution.

Trustworthy accounting. One question can incur routing, summarisation, answer and tool calls: record all of them. Normalise provider Token fields without counting cached input twice; retain call/source IDs, rate version and currency/FX provenance. Deduplicate imports and count tool fees once. Estimates are not provider balances or invoices; unknown usage stays unknown. Auto/activity statistics use App logs and comparisons use equal periods.

7 Open-source projects, tools and APIs

The Android app uses Java/XML and AndroidX components; the backend uses Python with SQLite. The following projects reduce infrastructure work so that development can focus on ModelPilot's context management, routing and user experience.

Project / library	What we use and what we will develop
Pydantic AI chat example MIT	We will adapt its FastAPI streaming and persisted-message patterns. Our additions are account/project scoping, cross-model context assembly, Auto routing and call accounting; the Android interface is developed separately.
FastAPI MIT	Already used for backend APIs. We will add chat, project, context and tool endpoints. Typed request validation and streaming support fit the existing Python service.
Room / MPAndroidChart Apache-2.0	Existing local database and chart libraries. We will extend Room tables for cached chats and summaries, and connect charts to model, project and time-filtered usage records.
Retrofit / Glide Apache-2.0 / BSD-2-Clause*	Already used by the forum: Retrofit exchanges typed HTTP data; Glide loads and caches news/post images. We will reuse them for ModelPilot's API endpoints and attachment/result previews.
AndroidX RecyclerView and SwipeRefreshLayout Apache-2.0	Existing forum list and refresh components. We will adapt list layouts, pagination and refresh/error states to the revised Explore design.
pypdf BSD-3-Clause	We will add text extraction for uploaded PDFs. Our code will select relevant passages, pass them into model context and link summaries to the original file.

*Glide includes additional licence notices for bundled components.

Model APIs. Extend the existing DeepSeek API connection and integrate Anthropic Messages as the second text API, subject to access. The backend will adapt request formats and usage fields. Image editing requires a separately validated API. API failures preserve the user's draft and allow retry or model reselection.

8 Development timeline: course weeks 4-15

The plan starts in week 4, with incremental integration and alpha/beta/final checkpoints subject to confirmed course dates. Optional modules must not delay the core assistant.

Weeks	Work to complete	Acceptance evidence
4-5	Freeze revised scope and contracts; build native Chat/project navigation; add account-scoped message persistence and one real model path.	Ask, receive, restart and reopen the same chat. Record the real call separately from sample data.
6-7	Add second provider, manual switching, versioned context builder and deterministic Auto; implement memory review/edit/reset.	Test A–B–A fact continuity under different context limits, corrected summaries and chat isolation; explain each Auto route.
8-9 Alpha	Integrate text-PDF tool, bounded tool loop, price-versioned call ledger and budget checks.	Complete a PDF document task end to end; reconcile all model/tool events; demonstrate timeout and budget recovery.
10-12 Beta	Connect Insights, activity/Auto logs, source-filtered import/export and price information. Add image editing if capability/access tests pass.	Charts trace to records; imports deduplicate; unknown fields remain unknown. An image result is a saved artefact, or the option stays unavailable.
13-14	Prioritise authentication, deletion, context regression, network recovery and accessibility. Integrate optional news/forum/game only after core tests pass.	Core isolation, continuity, rotation and offline tests pass. If extras are enabled, sources are linked and finance records remain isolated.
15 Final	Fix defects; freeze demo; finalise source archive, report, slides, MP4 video and attribution. Rehearse individual explanations.	Reproducible Android demonstration with test results, known limitations and member contributions.

Proposed team responsibilities. Zhang Li (24107757): accounts, persistence, pricing, budgets and Insights. Wang Tingdong (24107759): Chat/history, provider adapters and Auto/context/tools; supplementary forum/news. Liu Zongrun (24107745): native Chat/project/attachment UI, result presentation and device testing; supplementary game integration. All members review interfaces, test and record contributions.

9 Evaluation, risks and completion criteria

Core demonstration: create a project with two chats; summarise a text PDF with model A; continue with B and return to A using retained facts; inspect/correct memory; show Auto and manual selection, related usage and a budget limit with the draft preserved. Image editing, forum and game demonstrations are separate, optional additions and cannot substitute for core completion.

- **Context continuity:** use seeded facts/files in A–B–A tests; check relevant recall under a small context budget, summary edits/resets, deleted-source invalidation and separation of sibling chats/accounts. Record omissions and failures, not just successful examples.
- **Routing and usefulness:** use a fixed text/PDF suite (images if enabled). Check capabilities, reasons and preference effects; compare answer suitability, latency and total cost against a manual baseline. Do not assume Auto always saves money.
- **Correctness and privacy:** unit/contract tests for price versions, cache normalisation, tool-cost inclusion, import IDs and concurrent budget reservations. Test cross-account access, context separation and deletion. Imported daily buckets do not count as individual requests unless the source supplies a count.
- **Android quality:** test loading/error/empty states, restart, small screens, rotation, keyboard, large text and TalkBack. Measure responsiveness on a named device; separate provider latency from UI work. Run HTTP off the main thread; keep cached history readable.
- **Risk response:** quotas, outages or missing capabilities disable a route with a clear explanation; retries require cost awareness. Scanned PDFs/OCR, arbitrary skills and unverified providers are deferred. Retain the working text/PDF path if image or supporting modules remain incomplete.